Numerical Experiments with the Machine Learning Framework of Weinan E

Laura Nemeth

2026

Outline

Motivation

In 2017, Weinan E published "A Proposal on Machine Learning via Dynamical Systems," describing connections between dynamical systems, and the training of deep neural networks.

In this talk, we review the proposed framework and present numerical experiments illustrating how these ideas work in practice: we compare different numerical schemes and function representations for the control parameters, and perform error analysis for these methods.

Deep Neural Networks and Discrete Dynamical Systems

A deep neural network passes its input through a sequence of layers,

$$x_{k+1} = \sigma(A_k x_k + B_k), \qquad x_0 \in \mathbb{R}^d,$$

each a linear transform followed by a componentwise nonlinear activation σ. Our goal is to choose the weights $\{A_k, B_k\}$ so that the output x_K of our model approximates a target function g.

Taking the layer index k as time, a deep network can be seen as a discrete dynamical system. Choosing the weights by training is a control problem.

Residual Neural Networks and ODEs

A residual neural network is built from blocks of the form $x \mapsto x + f(x)$, where f is itself one or more layers of a neural network. In the case of a single layer,

$$x_{k+1} = x_k + \sigma(A_k x_k + B_k), \qquad x_0 \in \mathbb{R}^d.$$

Taking the layer index k as time, a residual neural network can be seen as a discretization of the ODE $x' = \sigma(A(t)x + B(t))$ using the Euler method.

Problem Formulation

We study the dynamical system

$$\frac{dx}{dt} = \sigma\big(A(t)\,x + B(t)\big), \qquad x(0) = a \in \mathbb{R}^d, \qquad t \in [0,1],$$

where $P(t) = (A(t), B(t))$ is the control and σ is a fixed nonlinear function.
We occasionally write $f(x, t;\, P) = \sigma(A(t)x + B(t))$ to emphasize that f depends on these control parameters.

For fixed P, the solution defines the flow map over the time interval $[0, 1]$:

$$\Phi_P^t : \mathbb{R}^d \to \mathbb{R}^d, \qquad a \mapsto x(t;\, a,\, P).$$

In practice, we may also map inputs in $\mathbb{R}^{d_{\text{in}}}$ to outputs in $\mathbb{R}^{d_{\text{out}}}$ using a pair of linear embeddings: $u \mapsto W_{\text{out}}\, \Phi_P^1(W_{\text{in}}\, u)$, though we will primarily focus on just the flow.

Training

The state $x(t, a, P)$ is defined implicitly by $F(x, a, P) = x(t) - a - \int_0^t f(x(s), s;\, P)\, ds$.
To approximate a target g, we minimize the error over a distribution μ of initial conditions,

$$E(P) = \int \|x(1; a, P) - g(a)\|^2 \, d\mu(a).$$

Setting $F(x(a, P), a, P) = 0$ and differentiating in P gives

$$\begin{bmatrix} D_1 F & D_2 F \end{bmatrix} \begin{bmatrix} D_P x \\ I \end{bmatrix} \quad \Longrightarrow \quad D_P x = -(D_1 F)^{-1} D_2 F.$$

And so the gradient is.

$$\nabla E(P)\, H = 2 \int \big(x(1; a, P) - g(a)\big)^\top D_P x(1; a, P)\, H \, d\mu(a).$$

We can then update the parameters via gradient descent.

Controllability Experiments

Neural networks are known to be very expressive: by the universal approximation theorem, a sufficiently large network can approximate any continuous function. However in practice, it can be very difficult to find a suitable control. We would like to better understand under what conditions gradient descent is likely to reach a good control.

Choice of Function Representation

The control $A(t)$ and $B(t)$ are functions on $[0, 1]$. In this work, we represent each entry by N values $\{y_k\}_{k=0}^{N-1}$ at fixed nodes and interpolate between the nodes. We compare two choices for interpolation:

- **Piecewise-linear spline**: values at uniformly spaced nodes using linear interpolation.
- **Chebyshev polynomial**: values at the Chebyshev nodes $t_k = \frac{1}{2}\left(1 - \cos\frac{k\pi}{N-1}\right)$, using Lagrange interpolation.

Choice of Function Representation

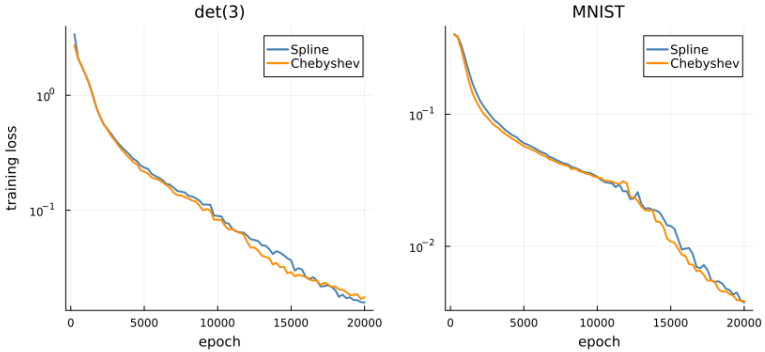
We compare the mean loss of two networks that differ only in their function representation. For each task, we train the model 100 times with a Spline representation and 100 times with a Chebyshev representation and compare the normalized error $\hat{E} = E/\mathrm{Var}$, where Var is the variance of the target data set.

Task	$\hat{E}$	$\hat{E}_c - \hat{E}_s$	99.9% CI
det(3), N=4	4.30×10^{-3}	$+0.55 \times 10^{-3}$	$[+0.15, +0.95] \times 10^{-3}$
det(3), N=8	2.52×10^{-3}	$+0.18 \times 10^{-3}$	$[-0.04, +0.40] \times 10^{-3}$
det(3), N=16	1.73×10^{-3}	-0.18×10^{-3}	$[-0.39, +0.03] \times 10^{-3}$
det(3), N=32	1.41×10^{-3}	-0.06×10^{-3}	$[-0.25, +0.13] \times 10^{-3}$
MNIST, N=4	6.81×10^{-3}	-0.31×10^{-3}	$[-0.87, +0.26] \times 10^{-3}$
MNIST, N=8	4.17×10^{-3}	$+0.35 \times 10^{-3}$	$[-0.25, +0.95] \times 10^{-3}$
MNIST, N=16	3.04×10^{-3}	-0.19×10^{-3}	$[-0.58, +0.20] \times 10^{-3}$
MNIST, N=32	2.44×10^{-3}	-0.30×10^{-3}	$[-0.49, -0.10] \times 10^{-3}$

For the remaining experiments, the Chebyshev representation is used.

Choice of Function Representation

The pooled training loss, comparing Spline against Chebyshev. For each task, the curve shows the per-epoch mean over the four different model depths.



ODE Solvers

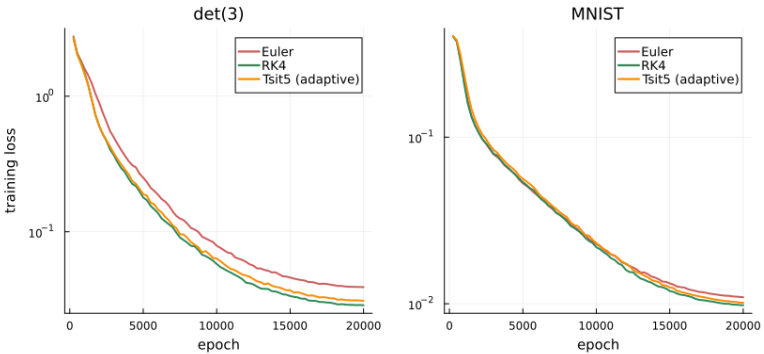
In our framework, a discrete neural net can be seen as the discretization of the model $\frac{dx}{dt} = \sigma\big(A(t)\,x + B(t)\big)$. But we can also experiment with other solvers. As before, we compare the normalized error $\hat{E} = E/\mathrm{Var}$, with Euler (the residual network) as the baseline.

Task	Solver	$\hat{E}$	$\hat{E} - \hat{E}_{\mathrm{euler}}$	99.9% CI
det(3)	Euler	5.69×10^{-3}	—	—
	RK4	4.24×10^{-3}	-1.45×10^{-3}	$[-2.03, -0.87] \times 10^{-3}$
	Tsit5 (adaptive)	4.75×10^{-3}	-0.95×10^{-3}	$[-1.57, -0.32] \times 10^{-3}$
MNIST	Euler	12.01×10^{-3}	—	—
	RK4	10.62×10^{-3}	-1.38×10^{-3}	$[-2.75, -0.02] \times 10^{-3}$
	Tsit5 (adaptive)	10.62×10^{-3}	-1.39×10^{-3}	$[-3.00, +0.22] \times 10^{-3}$

For the remaining experiments, we use the Runge-Kutta solver (RK4).

ODE Solvers

The pooled training loss under each solver. For each task the curve is the mean over all 100 trained models.



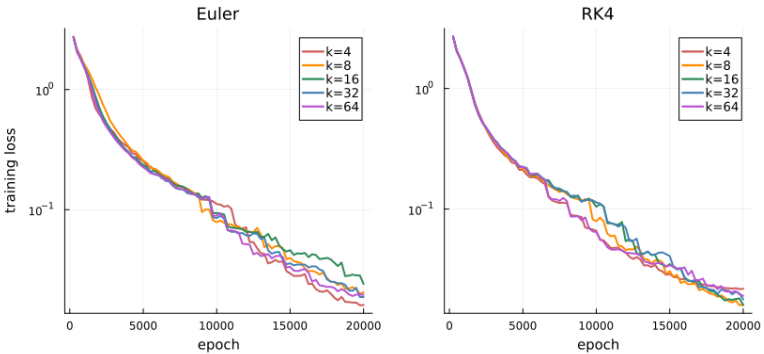
Solver Resolution

We can vary the number of timesteps k used by the ODE solver when computing the solution. We hold the number of control points fixed at $N = 8$. This adjusts the layer depth of the model without adjusting the number of model parameters. We train the model to approximate the 3x3 determinant.

Solver	k	$\hat{E}$	$\hat{E} - \hat{E}_{k=N}$	99.9% CI
Euler	4	4.9755	$+4.0996$	$[+3.6439, +4.5553]$
	8	0.8759	—	—
	16	0.0727	-0.8032	$[-0.9292, -0.6772]$
	32	0.0329	-0.8430	$[-0.9692, -0.7168]$
	64	0.0119	-0.8641	$[-0.9905, -0.7376]$
RK4	4	0.1079	$+0.0934$	$[+0.0797, +0.1072]$
	8	0.0145	—	—
	16	0.0054	-0.0091	$[-0.0114, -0.0067]$
	32	0.0027	-0.0118	$[-0.0139, -0.0096]$
	64	0.0028	-0.0117	$[-0.0138, -0.0095]$

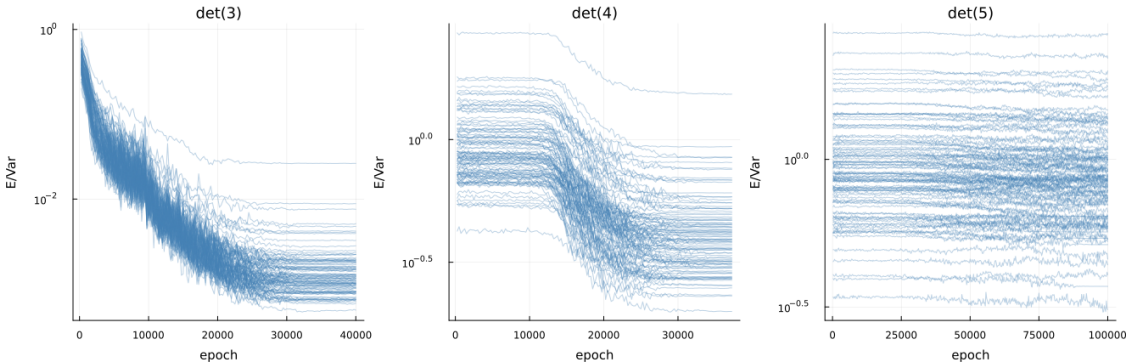
Solver Resolution

The pooled training loss at each resolution k, with the number of control points held fixed at N=8. For each k the curve is the mean over the 100 trained models.



Difficulties in Training the Determinant Function

From early experimentation, the determinant function appeared to be particularly difficult for the model to learn. For larger matrices, the model appears to be unable to learn the function via gradient descent.



Rather than getting stuck in a global minimum, the training seems to be stuck from the start regardless of initialization.

Gradient Calculation

In order to figure out what is happening, we calculate the gradient of the loss function when training on $g(M) = \det(M)$ explicitly. Let $M \in \mathbb{M}(n, n)$ be a random matrix with independent entries, $\mathbb{E}[M_{ij}] = 0$.

$$\begin{aligned}\nabla E(P) &= 2 \int_{\mathbb{M}} (x_P - \det(M)) \cdot \nabla_P x_P \, \mathrm{d}\mu \\ &= 2 \left(\int_{\mathbb{M}} x_P \cdot \nabla_P x_P \, \mathrm{d}\mu - \int_{\mathbb{M}} \det(M) \cdot \nabla_P x_P \, \mathrm{d}\mu \right)\end{aligned}$$

For the first term:

$$2 \int_{\mathbb{M}} (x_P \cdot \nabla_P x_P \, \mathrm{d}\mu) = \nabla_P \int_{\mathbb{M}} \|x_P\|^2 \, \mathrm{d}\mu = \nabla_P \|x_P\|_2^2$$

Gradient Calculation

Lemma: Let M be a random $n \times n$ matrix with independent entries and $\mathbb{E}[M_{ij}] = 0$. Recall that $\det(M) = \sum_{\pi \in S_n} \operatorname{sgn}(\pi) \prod_{i=1}^{n} M_{i\pi(i)}$, is a polynomial in the entries of M. For any polynomial $q(M)$ in the entries of M with $\deg q \leq n-1$,

$$\int_{\mathbb{M}} \det(M)\, q(M)\, \mathrm{d}\mu = \mathbb{E}\big[\det(M)\, q(M)\big] = 0$$

By linearity it suffices to take a monomial $q = \prod_{i,j} M_{ij}^{\beta_{ij}}$ with $\sum_{i,j} \beta_{ij} \leq n-1$. Expanding the determinant,

$$\mathbb{E}\big[\det(M)\, q\big] = \sum_{\pi \in S_n} \operatorname{sgn}(\pi)\, \mathbb{E}\Big[q \prod_{i=1}^{n} M_{i\pi(i)}\Big].$$

Since q has degree $\leq n-1$ at least one of the factors $M_{j\pi(j)}$ in $q \prod_i M_{i\pi(i)}$ is of multiplicity one.

Gradient Calculation

Since $M_{j\pi(j)}$ appears in neither q nor the remaining factors, it is independent of $R = q \prod_{i \neq j} M_{i\pi(i)}$, so the expectation factors:

$$\mathbb{E}\Big[q \prod_{i=1}^{n} M_{i\pi(i)}\Big] = \mathbb{E}\big[M_{j\pi(j)}\, R\big] = \mathbb{E}\big[M_{j\pi(j)}\big]\, \mathbb{E}[R] = 0,$$

because $\mathbb{E}[M_{j\pi(j)}] = 0$. Every term in the sum vanishes, and so

$$\mathbb{E}\big[\det(M)\, q(M)\big] = 0. \qquad \blacksquare$$

Gradient Calculation

Our model is the fixed point of the Picard iteration

$$x_0(t) = a, \qquad x_{k+1}(t) = a + \int_0^t \sigma\big(A\,x_k + B\big)\,\mathrm{d}s, \qquad a = W_{\mathrm{in}}\,\mathrm{vec}(M).$$

The first step of the iteration is: $x_1(t) = a + \int_0^t \sigma(Aa + B)\,\mathrm{d}s$. If σ is in C^n:

$$\sigma(Aa + B) = \sum_{d=0}^{n-1} \frac{\sigma^{(d)}(B)}{d!}(Aa)^d + R_n(Aa), \qquad R_n(Aa) = O\big(|Aa|^n\big)$$

so $x_1(1) = a + \sigma(B) + \sigma'(B)(Aa) + \frac{\sigma''(B)}{2}(Aa)^2 + \cdots + \frac{\sigma^{(n-1)}(B)}{(n-1)!}(Aa)^{n-1} + R_n(Aa)$.

Gradient Calculation

Given the form of $x_1(1)$, (or just using Taylor's theorem since $\sigma \in C^n$) we guess the form: $x_P = \sum_{d=0}^{n-1} X_d(a) + R_n(a)$, $R_n = O(|a|^n)$. With $z_0 = AX_0 + B$ and $h = AX_1 + AX_2 + \cdots$, substitute into the fixed point and expand σ about z_0:

$$\sum_d X_d(a) = a + \int_0^1 \left(\sigma(z_0) + \sigma'(z_0)\, h + \tfrac{1}{2}\sigma''(z_0)\, h^2 + \cdots \right) \mathrm{d}s$$

Matching degrees:

$$\begin{aligned}
\text{deg } 0: &\quad X_0 = \int_0^1 \sigma(z_0)\, \mathrm{d}s \\
\text{deg } 1: &\quad X_1 = a + \int_0^1 \sigma'(z_0)\, AX_1\, \mathrm{d}s \\
\text{deg } 2: &\quad X_2 = \int_0^1 \left[\sigma'(z_0)\, AX_2 + \tfrac{1}{2}\sigma''(z_0)(AX_1)^2 \right] \mathrm{d}s \\
&\ \ \vdots
\end{aligned}$$

Gradient Calculation

Each X_d appears on the right only through AX_d, sourced by lower degrees that each carry a factor of A, so for $d \geq 2$,

$$X_d = O\big(\|A\|^d\big).$$

Differentiating in P (which preserves the degree in a),

$$\nabla_P x_P = q(M) + \nabla_P R_n, \qquad \deg q \leq n-1, \quad \nabla_P R_n = O\big(\|A\|^{n-1}\big).$$

Gradient Calculation

Finally, by the Lemma, $\int_{\mathbb{M}} \det(M)\, q(M)\, \mathrm{d}\mu = 0$ since $\deg q \leq n-1$, leaving only the remainder:

$$\int_{\mathbb{M}} \det(M)\, \nabla_P x_P \, \mathrm{d}\mu = O\big(\|A\|^{n-1}\big).$$

$$\nabla E(P) = 2\left(\int_{\mathbb{M}} x_P \cdot \nabla_P x_P \, \mathrm{d}\mu - \int_{\mathbb{M}} \det(M) \cdot \nabla_P x_P \, \mathrm{d}\mu\right) = \nabla_P \|x_P\|_2^2 + O\big(\|A\|^{n-1}\big)$$

This, in combination with the vanishing gradient problem presents an issue. If we initialize A such that $\|A_k\| \approx 0$, then gradient descent pulls the model to approximate the zero function. If we initialize A with a larger norm, then the dynamics of the model are very likely to explode.

The Vanishing Gradient Problem

Training minimizes the approximation error, or loss $E = \|x_K - g(x_0)\|^2$, by gradient descent. By the chain rule,

$$\frac{\partial x_K}{\partial x_0} = \prod_{k=0}^{K-1} \Big(I + \sigma'(A_k x_k + B_k)\, A_k \Big), \qquad \left\| \frac{\partial x_{k+1}}{\partial x_k} - I \right\| \leq |\sigma'|\, \|A_k\|.$$

With many layers, this product is prone to either grow or decay exponentially, which leads to difficulty in training deep networks. This difficulty affects both discrete and continuous systems, but the continuous framework offers some tools with which we can attempt to address the issue.

To keep this product controlled, neural networks are usually initialized with $\frac{\partial x_{k+1}}{\partial x_k} \approx I$ or in the case of a residual neural net, with $\|A_k\| \approx 0$.

Failed Experiments

As we can see from the determinant, model initialization on its own is not enough to ensure that our gradient descent converges towards our target function in a reasonable amount of time for all target functions. I experimented with a number of techniques to help improve the training, however, none of them ended up helping:

- Sampling matrices from a biased set that does not have an expected value of zero. For example, random matrices centered around the identity rather than zero.
- Training on a simpler related problem simultaneously. For example, training the model to predict the matrix cofactors as well as the determinant. The idea being that the cofactors can be used to help calculate the determinant.
- Successively increasing the size of the matrix trained on. That is training on 2x2 matrices first, then 3x3, then 4x4, etc.

Future Research Directions

- Interrogating the training process itself: For this research, I leaned on and adapted generally accepted machine learning conventions for training a model, choosing to focus more on the structure of the models themselves.
- Expanding this framework to include other machine learning architectures. For example, looking at recurrent neural networks and LSTMs as an analogue of autonomous ODEs.
- Exploring the dynamics of more complicated nonlinearities in the model.

Exploring Model Architecture

For example, traditionally residual neural networks use a more complex nonlinearity than the one we've been exploring so far; at least two layers per step:

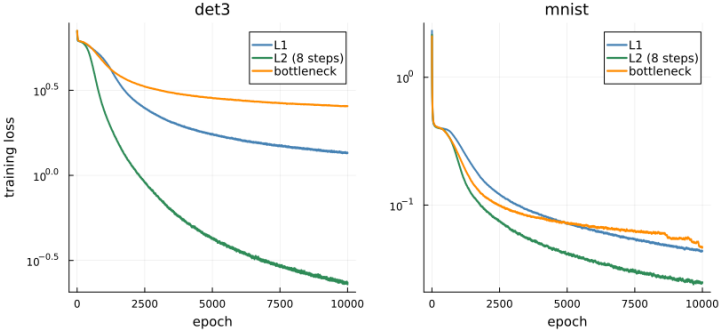
$$\frac{dx}{dt} = \sigma\big(A_2(t)\,\sigma(A_1(t)x + B_1(t)) + B_2(t)\big)$$

We compare the single-layer per-block model (L1), the two-layer per-block model (L2), and a "bottleneck" model with three layers per-block that contracts the dimension of the model, before expanding out again for the euler step.

Task	N	L1	L2	bottleneck
det(3)	1	0.416	0.075	0.803
	2	0.200	0.041	0.577
	4	0.138	0.019	0.201
	8	0.107	0.012	0.059
MNIST	1	0.084	0.067	0.105
	2	0.080	0.063	0.089
	4	0.073	0.059	0.075
	8	0.067	0.055	0.067

Exploring Model Architecture

The mean training loss over the 100 runs, for all three model architectures given on the previous slide.



References

- Weinan E, "A Proposal on Machine Learning via Dynamical Systems," *Commun. Math. Stat.*, 5:1–11, 2017.
- R.T.Q. Chen, Y. Rubanova, J. Bettencourt, and D. Duvenaud, "Neural Ordinary Differential Equations," *Advances in Neural Information Processing Systems* (NeurIPS), 2018.
- Weinan E, C. Ma, and L. Wu, "Machine Learning from a Continuous Viewpoint, I," *Sci. China Math.*, 63:2233–2266, 2020.
- Weinan E, C. Ma, S. Wojtowytsch, and L. Wu, "Towards a Mathematical Understanding of Neural Network-Based Machine Learning: What We Know and What We Don't," *CSIAM Trans. Appl. Math.*, 1(4):561–615, 2020.
- L.N. Trefethen, *Spectral Methods in MATLAB*, SIAM, 2000.
- C. Robinson, *Dynamical Systems: Stability, Symbolic Dynamics, and Chaos*, 2nd ed., CRC Press, 1998.

Choice of Nonlinearity

"Activation" functions are the nonlinear component in neural networks.

	$\sigma(x)$	$\sigma'(x)$	L_σ	Bounded	Smoothness
ReLU	$\max(0, x)$	$\begin{cases} 1 & x > 0 \\ 0 & x < 0 \end{cases}$	1	No	C^0
Sigmoid	$\frac{1}{1+e^{-x}}$	$\sigma(1-\sigma)$	$\frac{1}{4}$	$(0, 1)$	C^∞
Tanh	$\tanh(x)$	$\operatorname{sech}^2(x)$	1	$(-1, 1)$	C^∞

$\sigma : \mathbb{R} \to \mathbb{R}$ is applied componentwise to vectors: $\sigma(x) = (\sigma(x_1), \ldots, \sigma(x_d))^\top$.

Existence and Uniqueness

Theorem. The initial value problem $x' = f(x, t;\, P)$, $x(0) = a$ has a unique solution $x \in C([0,1]; \mathbb{R}^d)$.

Proof. $f(x, t;\, P) = \sigma(A(t)x + B(t))$ is a composition of two Lipschitz maps: $x \mapsto A(t)x + B(t)$ (constant $\|A(t)\|_{\mathrm{op}}$) and σ (constant L_σ). So $x \mapsto f(x, t;\, P)$ is Lipschitz with constant

$$L = L_\sigma \cdot \sup_{t \in [0,1]} \|A(t)\|_{\mathrm{op}} < \infty.$$

Define the Picard operator $(\mathcal{T}x)(t) = a + \int_0^t f(x(s), s;\, P)\, ds$.

A solution is a fixed point of $\mathcal{T}$. We will show $\mathcal{T}$ is a contraction on $C([0,1]; \mathbb{R}^d)$ and apply Banach's theorem.

Existence and Uniqueness

Let $\lambda > 0$. For $x \in C([0,1]; \mathbb{R}^d)$, define:

$$\|x\|_\lambda = \sup_{t \in [0,1]} e^{-\lambda t} \|x(t)\|.$$

Claim. $\|\cdot\|_\lambda$ is a norm on $C([0,1]; \mathbb{R}^d)$, equivalent to the sup norm.

Proof.
Positive definiteness. Since $e^{-\lambda t} > 0$ for all t, $\|x\|_\lambda = 0 \iff \|x(t)\| = 0$ for all $t \iff x = 0$, since $\|\cdot\|$ is a norm on $\mathbb{R}^d$.
Absolute homogeneity. $\|\alpha x\|_\lambda = \sup_t e^{-\lambda t} \|\alpha x(t)\| = |\alpha| \sup_t e^{-\lambda t} \|x(t)\| = |\alpha| \, \|x\|_\lambda$.
Triangle inequality.

$$\|x + w\|_\lambda = \sup_t e^{-\lambda t} \|x(t) + w(t)\| \le \sup_t e^{-\lambda t} \big(\|x(t)\| + \|w(t)\| \big) \le \|x\|_\lambda + \|w\|_\lambda.$$

Equivalence. $e^{-\lambda} \|x\|_\infty \le \|x\|_\lambda \le \|x\|_\infty$.

Existence and Uniqueness

We show $\mathcal{T}$ is a contraction in $\|\cdot\|_\lambda$. For any $x, w \in C([0,1];\mathbb{R}^d)$:

$$\|(\mathcal{T}x)(t) - (\mathcal{T}w)(t)\| = \left\| \int_0^t \big[f(x(s), s;\, P) - f(w(s), s;\, P)\big]\, ds \right\| \leq \int_0^t L\, \|x(s) - w(s)\|\, ds.$$

Rewriting $\|x(s) - w(s)\| = e^{\lambda s}\, e^{-\lambda s} \|x(s) - w(s)\| \leq e^{\lambda s}\, \|x - w\|_\lambda$:

$$\|(\mathcal{T}x)(t) - (\mathcal{T}w)(t)\| \leq L\, \|x - w\|_\lambda \int_0^t e^{\lambda s}\, ds = \frac{L}{\lambda}\, \|x - w\|_\lambda \left(e^{\lambda t} - 1\right).$$

Existence and Uniqueness

Multiplying both sides by $e^{-\lambda t}$ and taking the supremum over t:

$$\|\mathcal{T}x - \mathcal{T}w\|_\lambda \leq \frac{L}{\lambda} \|x - w\|_\lambda \left(1 - e^{-\lambda t}\right) \leq \frac{L}{\lambda} \|x - w\|_\lambda.$$

If $L/\lambda < 1$, then $\mathcal{T}$ is a contraction, and the Banach fixed point theorem gives a unique fixed point. □

Bounds on the Solutions

For the sigmoid and hyperbolic tangent activation functions, each component of σ is bounded by 1, so $\|f(x, t;\ P)\| \leq \sqrt{d}$. Since $t \in [0, 1]$,

$$\|x(t)\| \leq \|a\| + \sqrt{d}.$$

The solution is uniformly bounded, regardless of the value of the parameters.

Bounds on the Solutions

ReLU. Since $\|\sigma(A(t)x + B(t))\| \le \|A(t)x + B(t)\|$, we have $\|f(x, t;\, P)\| \le M\|x\| + \beta$ where $M = \sup_t \|A(t)\|_{\mathrm{op}}$ and $\beta = \sup_t \|B(t)\|$. Since $t \in [0, 1]$,

$$\|x(t)\| \le \|a\| + \int_0^t \big(M\,\|x(s)\| + \beta\big)\, ds.$$

Let $y(t) = \|x(t)\| + \beta/M$:

$$y(t) \le \Big(\|a\| + \tfrac{\beta}{M}\Big) + \int_0^t M \cdot y(s)\, ds.$$

By Gronwall's inequality, $y(t) \le \big(\|a\| + \beta/M\big)e^{Mt}$, hence

$$\|x(t)\| \le \Big(\|a\| + \tfrac{\beta}{M}\Big)e^{Mt} - \tfrac{\beta}{M}.$$

Continuous Dependence on Control Parameters

Theorem. Let $P_1 = (A_1, B_1)$ and $P_2 = (A_2, B_2)$ be two control parameters, and let x_1, x_2 be the corresponding solutions with the same initial condition a. Then

$$\|x_1(t) - x_2(t)\| \leq \frac{\delta}{L}\big(e^{Lt} - 1\big),$$

where $L = L_\sigma \cdot \sup_t \|A_1(t)\|_{\mathrm{op}}$ and $\delta = \sup_t \|f(x_2(t), t;\, P_1) - f(x_2(t), t;\, P_2)\|$.

Proof. Let $w(t) = x_1(t) - x_2(t)$. If we write this as an integral:

$$\begin{aligned} w(t) &= \int_0^t \big[f(x_1(s), s;\, P_1) - f(x_2(s), s;\, P_2)\big]\, ds \\ &= \int_0^t \big[f(x_1, s;\, P_1) - f(x_2, s;\, P_1) + f(x_2, s;\, P_1) - f(x_2, s;\, P_2)\big]\, ds \\ &= \int_0^t \big[f(x_1, s;\, P_1) - f(x_2, s;\, P_1)\big]\, ds + \int_0^t \big[f(x_2, s;\, P_1) - f(x_2, s;\, P_2)\big]\, ds. \end{aligned}$$

Continuous Dependence on Control Parameters

The first bracket is bounded by $L\,\|w(s)\|$, the second by δ. So:

$$\|w(t)\| \le \int_0^t L\,\|w(s)\|\,ds + \delta\,t.$$

Let $y(t) = \|w(t)\| + \delta/L$. Then,

$$y(t) \le \frac{\delta}{L} + L\int_0^t y(s)\,ds.$$

By Gronwall's inequality, $y(t) \le (\delta/L)\,e^{Lt}$, hence

$$\|w(t)\| \le \frac{\delta}{L}\big(e^{Lt} - 1\big). \quad \square$$